

Prediksjon av lengde på sykehusopphold

Prosjektrapport INF161

ANKITA JOHAR

Innholdsfortegnelse

Prediksjon av lengde på sykehusopphold	0
<i>Prosjektrapport INF16.....</i>	<i>Feil! Bokmerke er ikke definert.</i>
<i>Formål</i>	<i>2</i>
<i>Del 1: Datatilberedning og utforskende dataanalyse</i>	<i>3</i>
Forberedelse av data	4
Binære variabler	5
Fjerning av kolonner	5
Ferdig forberedt data	7
Utforskende dataanalyse	7
<i>DEL 2: Modellering og prediksjon</i>	<i>17</i>
Håndtering av manglende data	17
Feature engineering.....	18
Modellering og model valg	19
Generaliseringsevne.....	21
Feature importance.....	22
Modell valg for nettside	23
<i>DEL 3: Drøfting av resultater og mulige forbedringer</i>	<i>24</i>

Formål

Sykehusopphold for kritisk syke pasienter er ofte preget av usikkerhet og kompleksitet, både for pasientene selv og for helsevesenet som behandler dem. Hvor lenge en pasient må bli på sykehuset, avhenger av mange faktorer, alt fra pasientens alder og helsetilstand til den underliggende sykdommens alvorlighetsgrad og hvilke fysiologiske verdier de har ved innleggelsen. Å forutsi hvor lenge en pasient vil trenge sykehusopphold kan gi helsepersonell verdifull innsikt i ressursbehov, bemanning og pasientflyt. Dette er noe som kan bidra til mer effektiv drift og bedre pasientopplevelser.

Målet med dette prosjektet er å bygge en maskinlæringsmodell som kan anslå sykehusoppholdet for nye pasienter. Ved hjelp av historisk data fra 8261 pasienter, inkludert demografiske opplysninger, sykdomskategorier og fysiologiske målinger, vil vi utforske hvilke faktorer som sterkest påvirker oppholds lengden. Prosjektet innebærer flere steg, fra forberedelse og rensing av data, til modellutvikling og testing av ulike algoritmer. Alt med fokus på å finne den modellen som presterer best.

Del 1: Datatilberedning og utforskende dataanalyse

I dette prosjektet benyttes fire datasett – tre i CSV-format og ett i JSON-format , som til sammen dekker sykehusdata, fysiologiske målinger, demografiske opplysninger og informasjon om sykdomsalvorlighet. Datasettene omfatter totalt 8261 kritisk syke pasienter fra fem medisinske sentre i USA, med data fra periodene 1989-1991 og 1992-1994.

Formålet med datatilberedningen er å strukturere rådataene slik at de kan inngå som en samlet data-frame i en maskinlæringsmodell. Det viktigste med datatilberedningen er å hente ut relevante kolonner, sørge for at de ulike kolonnene har riktig datatype og fjerne eventuelle feil og ufullstendig data.

Den utforskende dataanalysen bygger videre på denne tilberedningen ved å utnytte både visualiseringer og statistiske analyser, for å avdekke mønstre og sammenhenger i dataene. Gjennom den utforskende dataanalysen får vi innsikt i hvordan dataene fordeler seg, eventuelle uteliggere og hvor det finnes manglende verdier. Denne innsikten danner grunnlaget for videre bearbeiding av datasettene, eksempelvis gjennom feature engineering, hvor nye variabler kan utvinnes, kombineres eller fjernes basert på datamønstre og faglige vurderinger. Slik kan vi tilrettelegge for at modellen benytter de mest relevante og informative variablene.

For å ivareta dataintegriteten deler vi opp datasettene i trenings, validerings og testdata, før den utforskende dataanalysen, der all analyse og bearbeiding utføres kun på treningsdataene. Dette er viktig for å sikre at beslutninger om feature engineering og modelljusteringer ikke påvirkes av informasjon fra validerings eller testdata, slik at modellen senere kan evalueres på data den ikke har sett før.

Forberedelse av data

For å starte datatilberedningen henter vi inn de fire datasettene: demographic.csv, hospital.csv, physiological.txt og severity.json. Målet er å samle disse datasettene i en data-frame som kan brukes i maskinlæringsmodellen. For å få til dette, bearbeides blant annet severity.json ved hjelp av funksjonen «Explode», slik at informasjonen fra alle datasettene kan samles og struktureres i en data-frame.

Etter å ha kombinert alle datasettene i en data-frame, fjernet eventuelle duplikater for å sikre datakvalitet og redusere risikoen for bias. Deretter delt dataene inn i trenings, validerings og testdata, kan vi begynne å analysere kolonnene og få en oversikt over variablenes struktur og innhold. Etter innlesing av data, ser kolonnene slik ut:

pasient_id	object
alder	float64
kjønn	object
utdanning	float64
inntekt	object
etnisitet	object
sykehusdød	int64
oppholdslengde	int64
blodtrykk	float64
hvite_blodlegemer	float64
hjerterefrekvens	float64
respirasjonsfrekvens	float64
kroppstemperatur	float64
lungefunksjon	float64
serumalbumin	float64
bilirubin	float64
kreatinin	float64
natrium	float64
blod_ph	float64
glukose	float64
blodurea_nitrogen	float64
urinmengde	float64
sykdomskategori_id	object
sykdomskategori	object
dødsfall	object
sykdom_underkategori	object
antall_komorbiditeter	object
koma_score	object
adl_pasient	object
adl_stedfortreder	object
fysiologisk_score	object
apache_fysiologisk_score	object
overlevelsesestimat_2mnd	object
overlevelsesestimat_6mnd	object
diabetes	object
demens	object
kreft	object
lege_overlevelsesestimat_2mnd	object
lege_overlevelsesestimat_6mnd	object
dnr_status	object
dnr_dag	object
dtype:	object

Figure 1: Oversikt over kolonner

Flere kolonner har typen «object», noe som ofte betyr tekststrenger, manglende verdier, eller blandede datatyper. Dette kan være problematisk i modelltrening som krever numeriske verdier. Kategoriske variabler omgjøres, som kjønn, inntekt, etnisitet, sykdomskategori, kreft og dnr-status, til binære variabler (dummies) for å sikre at de fungerer i modellen.

I tillegg ble kolonner som pasient_id, dødsfall, antall komorbiditeter, diabetes og demens definert som heltall, mens variabler som koma_score, adl_pasient og ulike overlevelsesestimater ble konvertert til flyttall. Disse datakonverteringene sikrer konsistens og optimaliserer både datakvalitet og minnebruk, noe som reduserer risikoen for feil i analysen og modelleringen.

Binære variabler

Når vi omgjør kategoriske variabler til numeriske formater, er det viktig å velge riktig metode, da dette kan påvirke hvordan modellen forstår dataene. I dette prosjektet velges to ulike tilnærminger, OneHotEncoder og mapping, basert på variablenes egenskaper.

For variablene kjønn, etnisitet og sykdomskategori brukes OneHotEncoder fordi disse er nominale variabler uten naturlig rekkefølge. Å gi dem numeriske verdier som 1, 2, 3, kan mislede modellen til å tro at det finnes en rangering at en kategori er 'høyere' eller 'lavere', noe OneHotEncoder unngår ved å lage en egen kolonne for hver kategori. Dette sikrer at modellen ikke feiltolker relasjoner mellom kategoriene.

For variablene inntekt, kreft og dnr-status brukes derimot mapping. Inntekt har en naturlig rekkefølge f.eks. lav, middels, høy og kreft og dnr-status er binære (ja/nei). Ved å tildele dem stigende numeriske verdier bevarer vi rangeringen eller binærstrukturen, og modellen kan tolke disse variablene effektivt uten behov for ekstra kolonner.

Riktig valg av metode for hver variabel gjør dataene lettere å tolke og minimerer risikoen for feiltolkning i modellen

Fjerning av kolonner

En viktig grunn til å fjerne enkelte kolonner er basert på domenekunnskap. Innen medisinske datasett kan spesialkunnskap bidra til å identifisere variabler som gir begrenset verdi for analysen. I tillegg fjerner vi kolonner med lav variasjon eller informasjonsverdi, da variabler som har samme verdi for de fleste observasjoner tilfører lite nytt for modellen og kan redusere effektiviteten.

Ettersom målet med prosjektet er å predikere pasientens oppholdslengde fra dag 1, er det hensiktsmessig å fjerne variabler som måles senere enn dag 1, slik som bilirubin og adl_stedfortreder. Disse representerer informasjon som ikke er tilgjengelig ved innleggelsen og kan føre til skjevhet i prediksjonene dersom de ble inkludert. Slik sikrer vi at modellen kun baserer seg på data som realistisk kan brukes fra første dag. Videre gir det mening å fjerne sykehusdød og dødsfall, ettersom disse beskriver pasientens endelige utfall og overlapper med det modellen prøver å predikere. Hvis disse beholdes, kan de virke som en "snarvei" og påvirke modellens fokus på faktorer som er tilgjengelige tidlig i pasientforløpet, noe som gir mer pålitelige prediksjoner. Vi valgte også å fjerne sykdomskategori_id og sykdomskategori, etter at en analyse viste at de var identiske. En sammenligning av verdiene i disse kolonnene viste fullt samsvar, noe som gjorde det unødvendig å beholde begge.

```
: df_train['sykdomskategori_id'].value_counts(dropna=False)

: sykdomskategori_id
A1s      2518
BrY      1701
ChE       841
DWw       358
Name: count, dtype: int64

: df_train['sykdomskategori'].value_counts(dropna=False)

: sykdomskategori
ARF/MOSF      2518
COPD/CHF/Cirrhosis  1701
Cancer         841
Coma           358
Name: count, dtype: int64
```

Figur 2: Sykdom sammenheng

```
sykdom_map = {
    'A1s': 'ARF/MOSF',
    'BrY': 'COPD/CHF/Cirrhosis',
    'ChE': 'Cancer',
    'DWw': 'Coma'
}

print(all(df_train['sykdomskategori_id'].map(sykdom_map) == df_train['sykdomskategori']))

True
```

Figur 3: Sykdom sammenheng 2.0

I tillegg vurderes sykdom_underkategori og fant at denne ga mer spesifikk informasjon. Siden hver underkategori er knyttet til en spesifikk overkategori, for eksempel at ChE i sykdomskategori_id representerer ulike krefttyper som lung eller colon. Derfor ble sykdom_underkategori ansett som mest informativ. Dermed kunne vi trygt fjerne sykdomskategori_id og sykdomskategori.

```

: #sjekker verdi overlapp mellom id og underkategori
sammenheng = df_train.groupby('sykdomskategori_id')['sykdom_underkategori'].unique()
print(sammenheng)

sykdomskategori_id
A1s      [ARF/MOSF w/Sepsis, MOSF w/Malig]
BrY      [COPD, CHF, Cirrhosis]
ChE      [Lung Cancer, Colon Cancer]
DWw      [Coma]
Name: sykdom_underkategori, dtype: object

```

Figur 4: Sykdom sammenheng 3.0

For å forberede sykdom_underkategori for modellering omgjøres denne kolonnen til flere binære variabler ved bruk av OneHotEncoder, slik at hver underkategori ble representert med egne kolonner med verdiene 0 eller 1. Denne metoden gir modellen mulighet til å tolke de forskjellige sykdomstypene, uten å feiltolke dem som rangert, noe som gir en mer nøyaktig representasjon i analysen og modelleringen.

Ferdig forberedt data

Etter datatilberedningen er alle datasettene samlet i en data-frame, der alle kolonner har fått riktig datatype. Dette sikrer at modellen kan tolke og behandle dataene på riktig måte og at vi unngår potensielle feil knyttet til inkonsistente datatyper. For å forberede dataene til modellering delte vi dataen inn i trenings, validerings og testdata.

Treningsdata (X_train og y_train) brukes til å bygge og tilpasse modellen, valideringsdata (X_val og y_val) for å evaluere og finjustere modellens ytelse, mens testdata (X_test og y_test) holdes til side for en endelig evaluering av modellens generaliserbarhet.

Denne oppdelingen er også viktig for utforskende dataanalyse, der analysen hovedsakelig utføres på treningssettet. Dette sikrer at eventuelle mønstre eller innsikt som avdekkes i den utforskende datanalysen, ikke skaper skjevhet i modellvalideringen eller testing.

Utforskende dataanalyse

Den utforskende dataanalysen gir en innsikt i datasettet ved å undersøke variablenes fordeling, sammenhenger og potensielle uteliggere. Gjennom visualiseringer og statistiske oppsummeringer kan vi bedre forstå strukturen i dataene og identifisere mønstre som kan påvirke modellens ytelse.

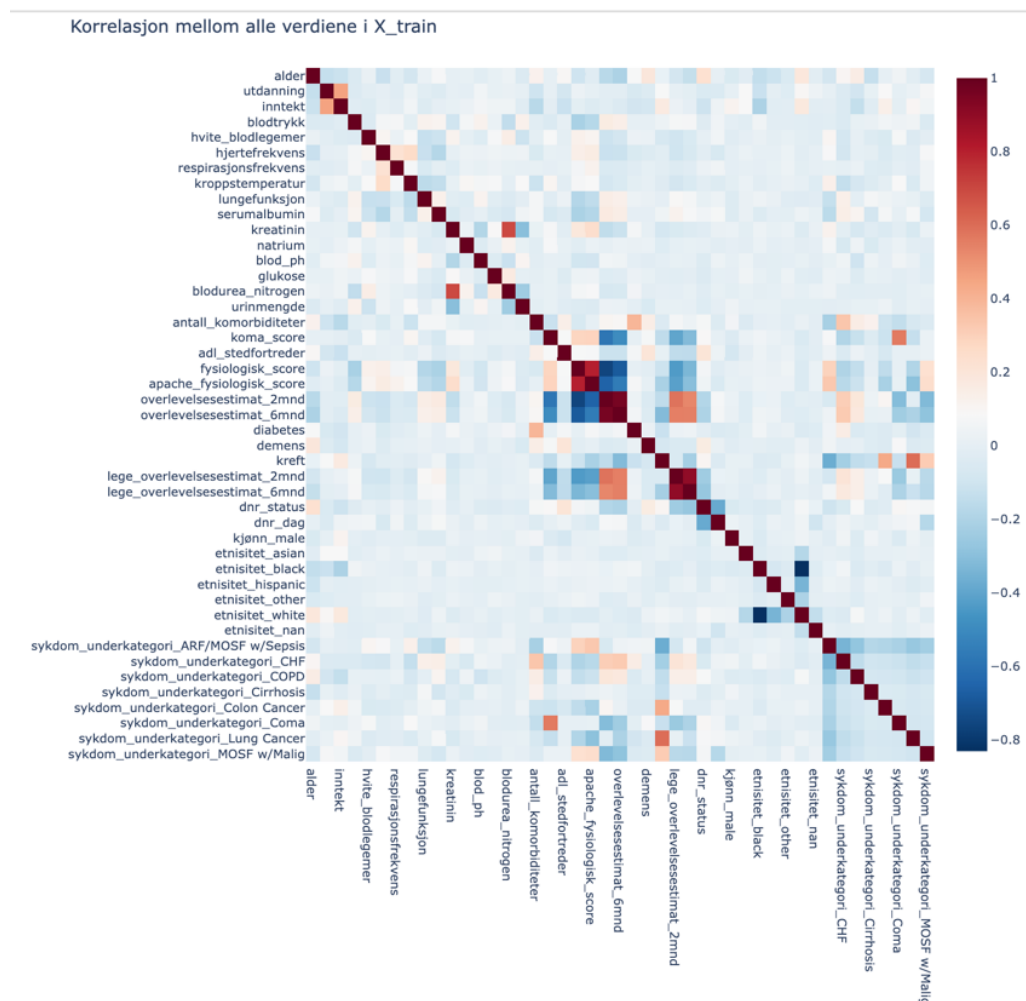
I den innledende analysen ble dataene undersøkt for å få en generell forståelse av variablene og deres statistiske egenskaper. Et interessant funn er at minimumsalderen

har en uventet verdi på -1, noe som sannsynligvis skyldes en feil i datainnsamlingen. Dette bør korrigeres før modellering.

	alder	utdanning	inntekt	blodtrykk	hvite_bloedlegemer	hjerterefrekvens	respirasjonsfrekvens	kroppstemperatur	lungefunksjon	serumal
count	5418.000000	4434.000000	3653.000000	5418.000000	5300.000000	5418.000000	5418.000000	5418.000000	4067.000000	3440.0
mean	62.698942	11.661479	0.913496	84.216039	12.331104	97.226412	23.400886	37.108185	239.822272	2.9
std	15.771799	3.457808	1.028970	27.799628	8.985583	31.792115	9.598255	1.262383	109.944850	0.9
min	-1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	32.093750	12.000000	0.7
25%	52.998990	10.000000	0.000000	63.000000	7.000000	72.000000	18.000000	36.093750	156.000000	2.3
50%	65.063445	12.000000	1.000000	77.000000	10.599609	100.000000	24.000000	36.695310	223.781250	2.8
75%	74.152220	13.000000	2.000000	107.000000	15.398438	120.000000	28.000000	38.195310	304.750000	3.5
max	101.847960	31.000000	3.000000	193.000000	117.000000	232.000000	90.000000	41.695310	869.375000	29.0

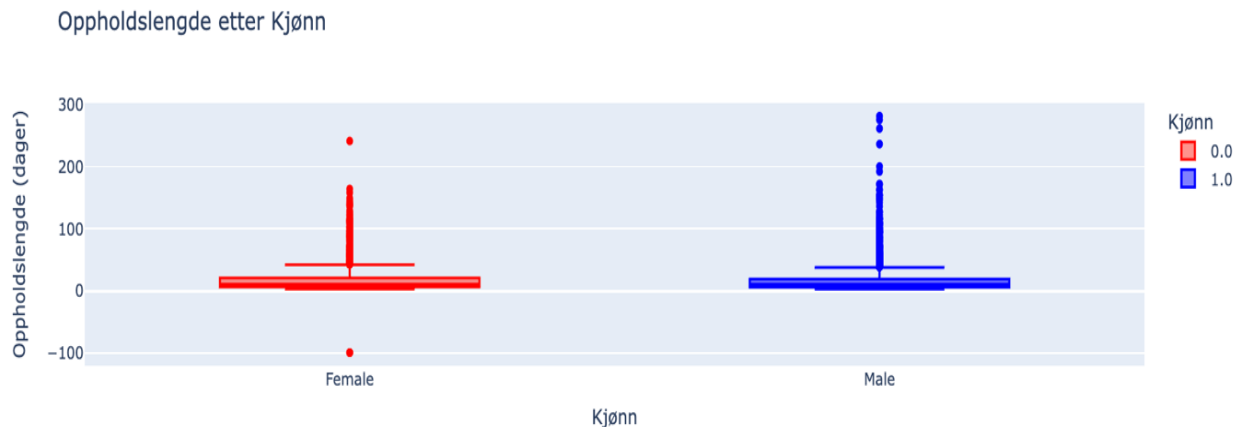
Figur 5: Statistisk oppsummering (kun en liten del av tabellen)

Korrelasjonsmatrisen på treningsdata viste at noen variabler har høy korrelasjon med hverandre. Eksempelvis er det høy korrelasjon mellom variabler som fysiologisk_score og apache_fysiologisk_score noe som kan indikere redundans. Slik korrelasjon kan påvirke modellens ytelse, og vi bør vurdere å fjerne eller kombinere sterkt korrelerte variabler. Dette kommer vi tilbake til.

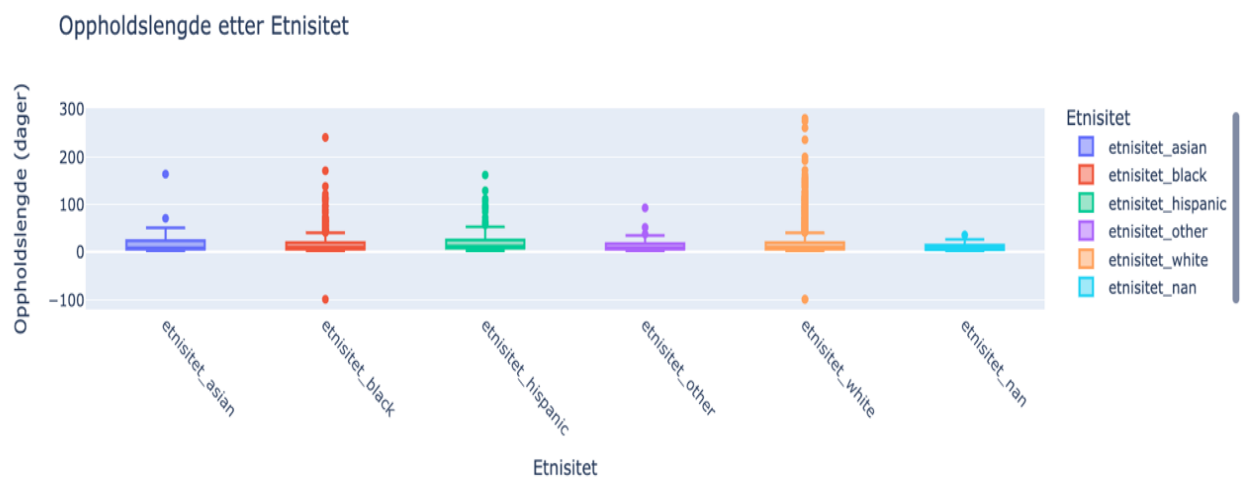


Figur 6: Korrelasjon X_train

Visualiseringen av kjønnsfordelingen viser fordelingen mellom mannlige og kvinnelige pasienter. I tillegg ble det funnet variasjoner i oppholdslengde mellom kjønnene, som kan være en viktig faktor for modelleringen. For etnisitetsfordelingen ble det observert en overvekt av hvite pasienter, og oppholdslengden varierte mellom de forskjellige etnisitetsgruppene. Dette kan indikere at demografiske faktorer som kjønn og etnisitet har en viss sammenheng med oppholdslengde.

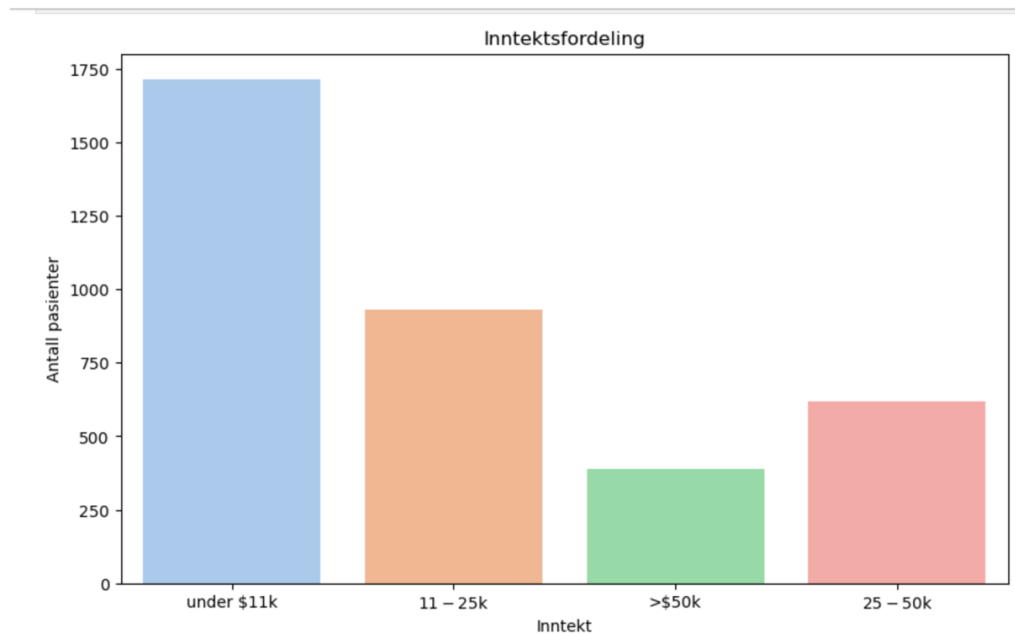


Figur 7: Oppholdslengde etter kjønn

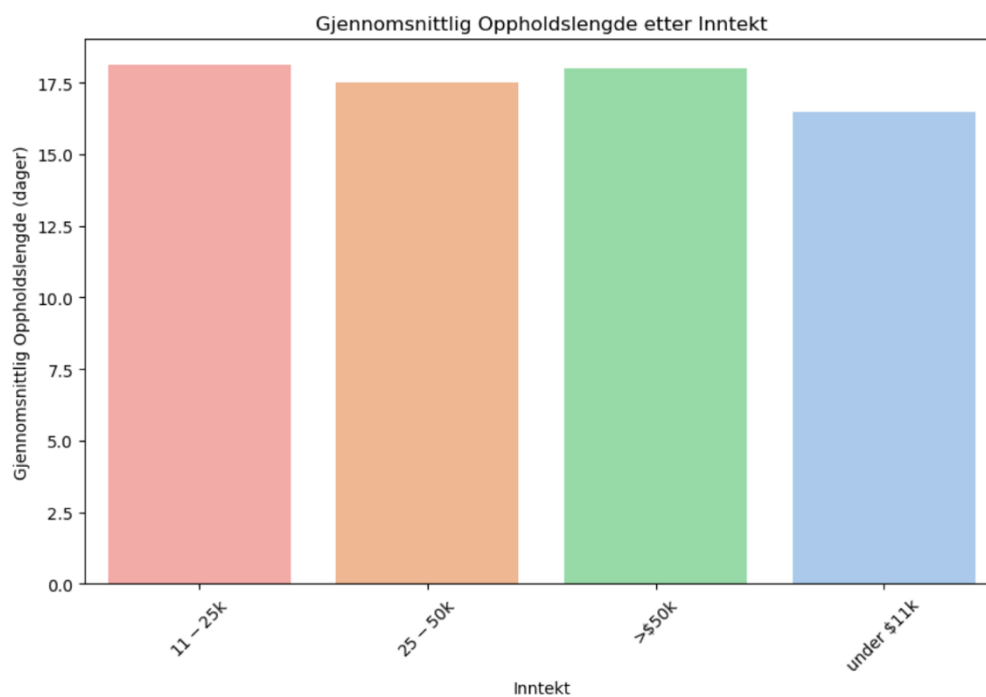


Figur 8: Oppholdslengde etter etnisitet

Inntektsfordelingen blant pasientene ble også analysert. De fleste pasienter hadde en inntekt under \$11 000, og visualiseringene viste at oppholdslengde varierer noe med inntektsnivå. Pasienter med høyere inntekt hadde generelt lengere opphold, noe som kan indikere at økonomiske forhold påvirker pasientens tilgang til helsetjenester og behandlinglengde.

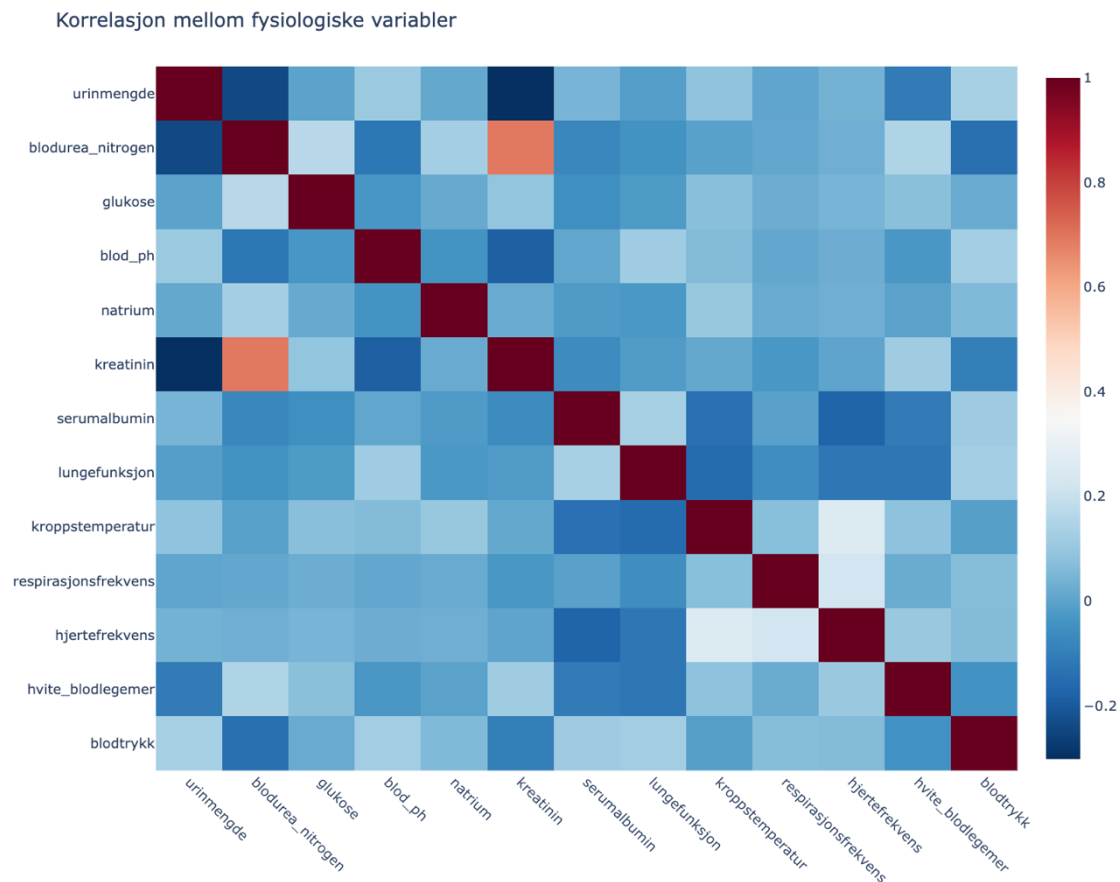


Figur 9: Inntektsfordeling

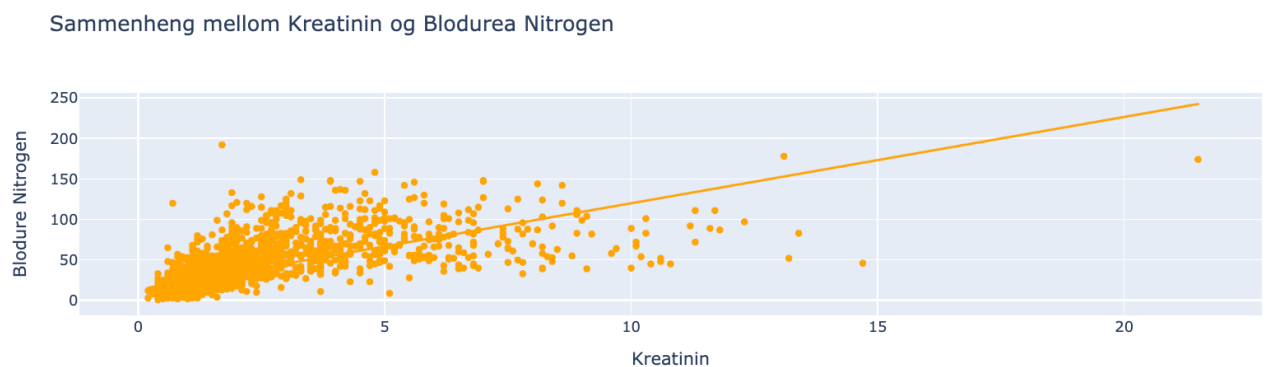


Figur 10: Oppholdslengde etter inntekt

For å identifisere de viktigste fysiologiske variablene for prediksjon av oppholdslengde, analyseres korrelasjoner mellom dem. Vi ser blant annet en sterk korrelasjon mellom blodurea nitrogen og kreatinin. Høyere nivåer av kreatinin sammenfaller med høyere nivåer av blodurea nitrogen. Siden begge måler nyrefunksjon, kan det være nyttig å forenkle dataen ved å beholde kun en av dem for å unngå redundans.

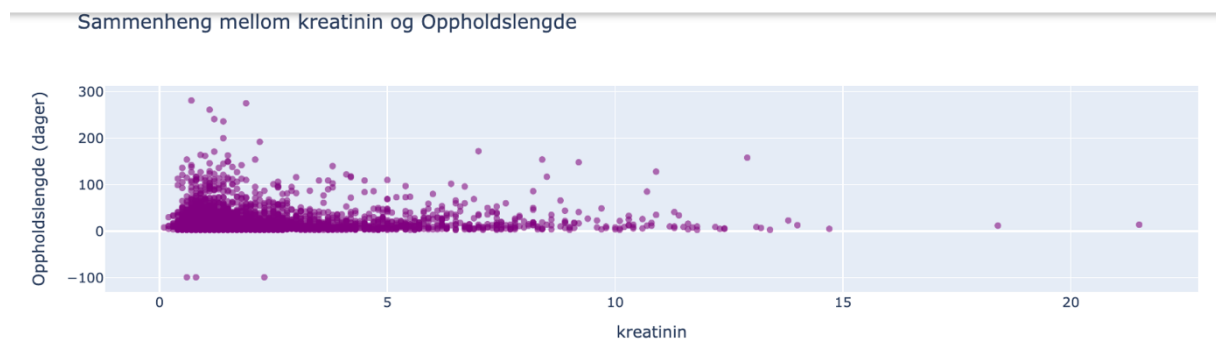


Figur 11: Korrelasjon mellom fysiologiske variabler

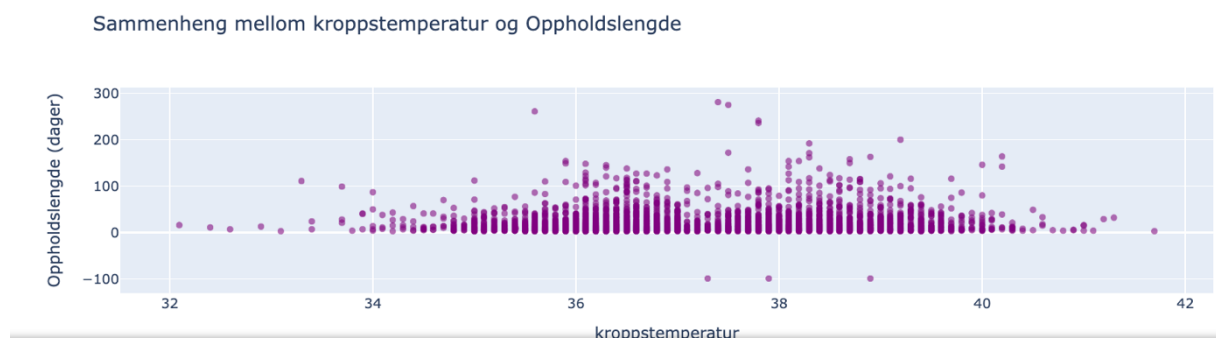


Figur 12: Sammenheng mellom Kreatinin og Blodurea Nitrogen

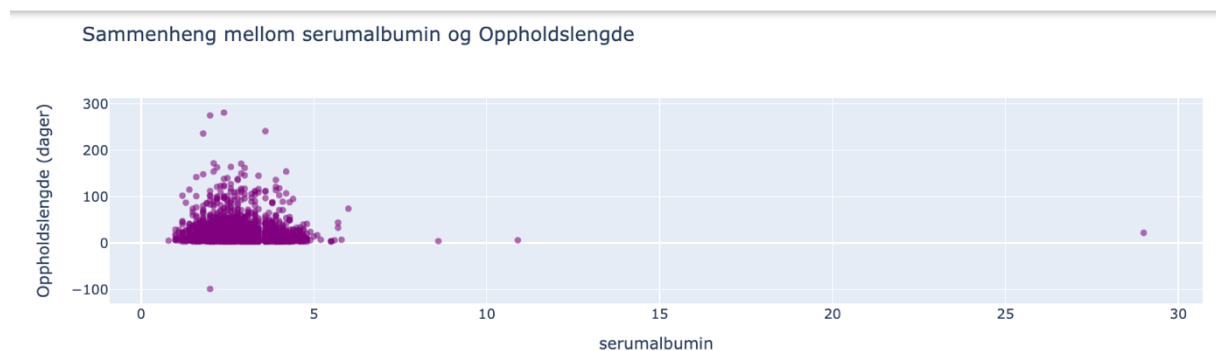
For å undersøke sammenhengen mellom ulike fysiologiske variabler og oppholdslengde ble det utført en spredningsplott-analyse, der oppholdslengde ble plottet mot hver variabel i datasettet. Gjennom analysen kunne vi identifisere om det eksisterte tydelige lineære eller ikke-lineære mønstre mellom disse variablene og oppholdslengde. Visualiseringene som presenteres under er de som viste størst innsikt i forhold til oppholdslengde, og som derfor kan gi en pekepinn om hvilke fysiologiske mål som er særlig viktige for å forutsi pasientenes oppholdstid.



Figur 13: Pasienter med kreatinin nivå mellom 0.5 og 2 har lengere oppholdstid

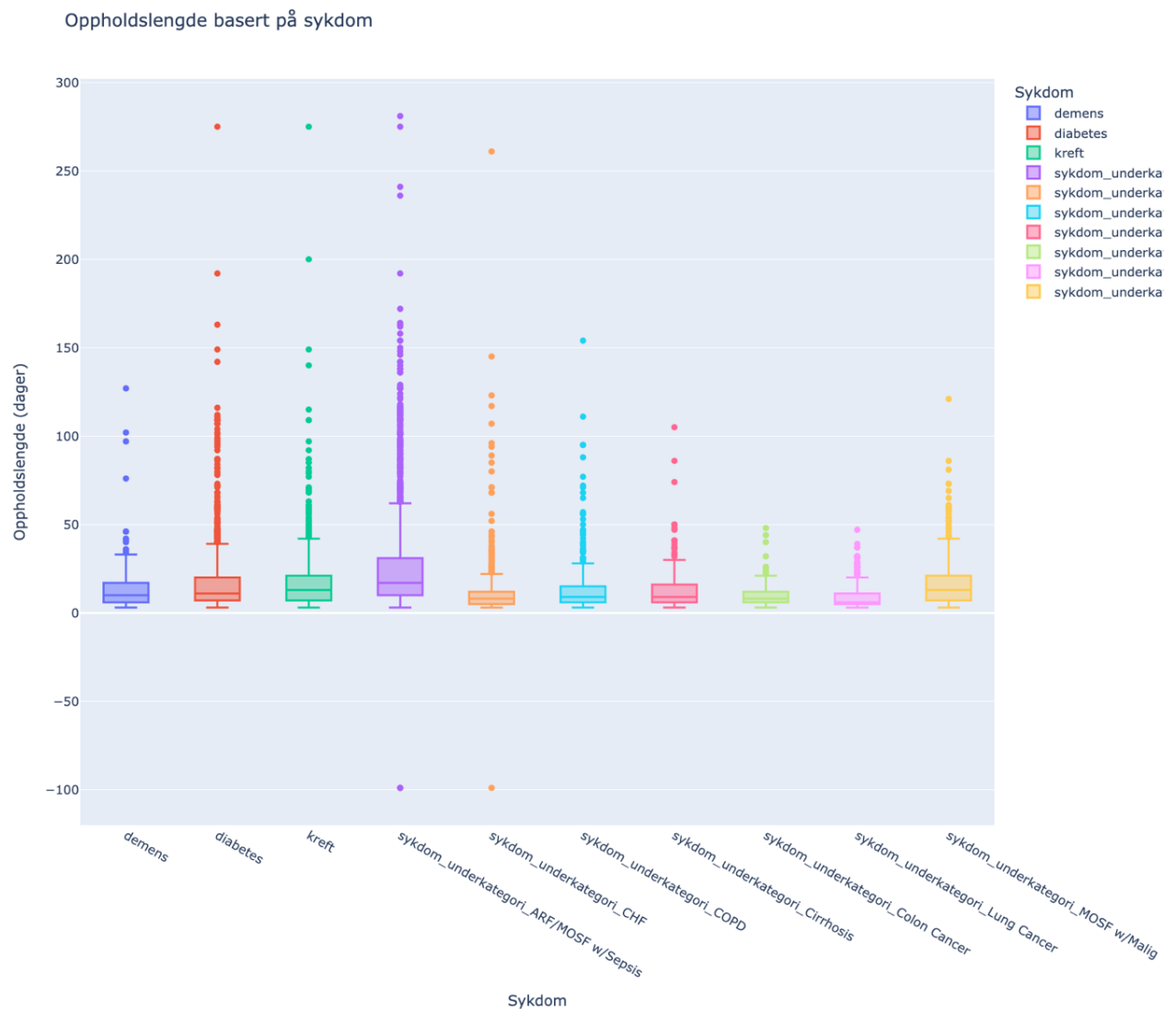


Figur 14: Pasienter med lavere eller høyre kroppstemperatur enn 37/37.5 °C (normal temperatur) har lengere oppholdstid



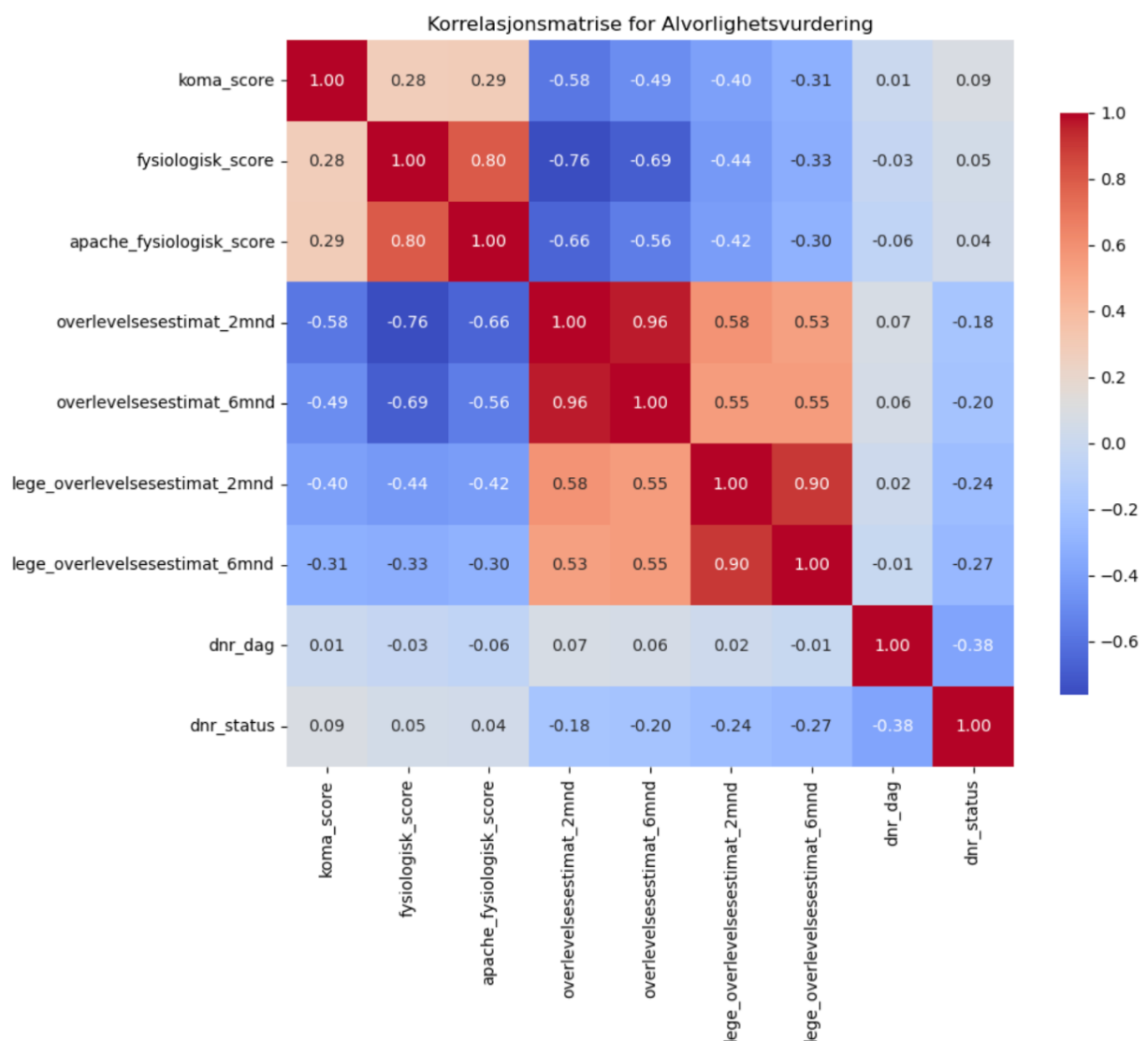
Figur 15: Pasienter med serumalbumin mellom 2 og 3 har høyere oppholdstid

I analysen av ulike sykdomsvariabler undersøkes det hvordan spesifikke sykdommer og tilstander, som demens, diabetes, kreft, og andre alvorlige sykdomskategorier, påvirker oppholdslengden. Denne innsikten kan hjelpe oss å identifisere hvilke sykdomskategorier som har størst påvirkning på oppholdslengden og kan derfor være verdifulle variabler i modelleringen. Figuren under viser at sykdomsunderkategorien ARF/MOSF med sepsis har flest pasienter med lengst oppholdslengde



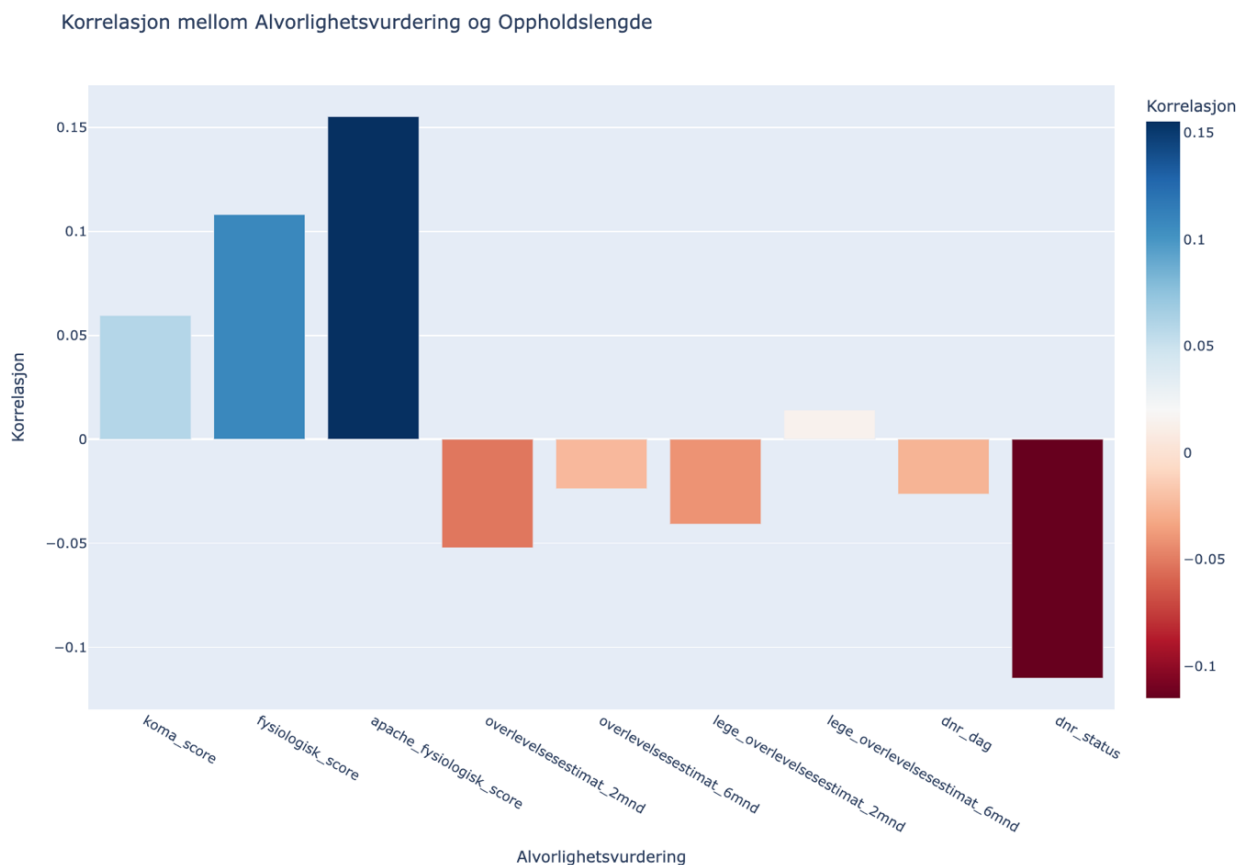
Figur 16: Oppholdslengde basert på sykdom

For å undersøke sammenhengen mellom ulike alvorlighetsvurderinger og deres relasjon til oppholdslengde, opprettes en korrelasjonsmatrise. Vi ser at det er høy korrelasjon mellom `fysiologisk_score`, `apache_fysiologisk_score`, `overlevelsesestimert_2mnd`, `lege_overlevelsesestimert_2mnd`, og `overlevelsesestimert_6mnd`. Denne høye korrelasjonen tyder på at disse variablene måler lignende aspekter av pasientens tilstand. Slike innsikter kan være nyttige for å redusere redundans i dataene og forenkle modellens inputvariabler. Dette gir også verdifull informasjon om hvilke faktorer som har størst betydning for prediksjonen av oppholdslengde.



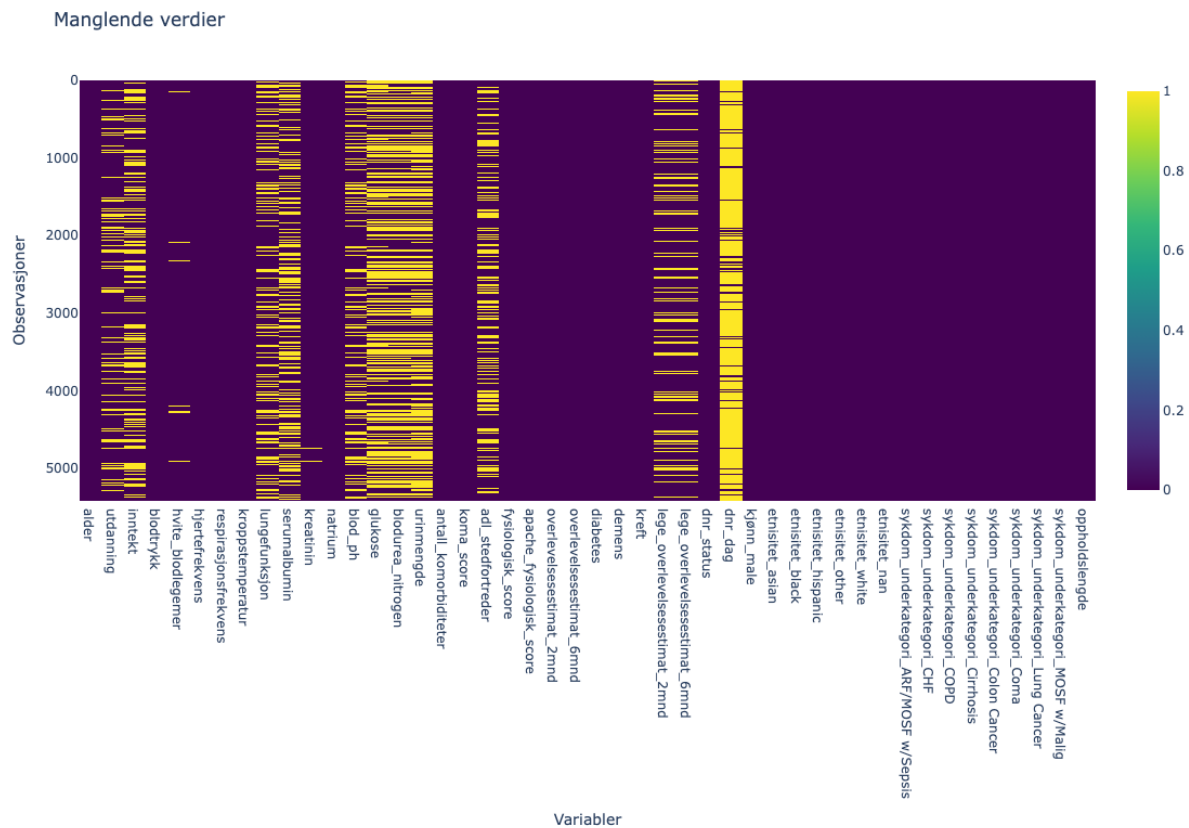
Figur 17: Korrelasjon mellom alvorlighetsvurdering

Videre analyseres sammenhengen mellom ulike alvorlighetsvurderinger og oppholdslengde ved å beregne korrelasjonen mellom hver vurderingsvariabel og oppholdslengden. Vi observerer spesielt høy korrelasjon mellom fysiologisk_score og oppholdslengde samt en spesielt lav korrelasjon mellom dnr_staus og oppholdslengde. Dette kan være nyttig informasjon for feature engineering, der vi kan vurdere hvilke variabler som er nødvendige i modellen.



Figur 18: Korrelasjon mellom oppholdslengde og alvorlighetsvurdering

Forekomsten av manglende verdier i datasettet undersøkes ved hjelp av et heatmap som visualiserer hvilke variabler og observasjoner som har nullverdier. Denne analysen gir en oversikt over hvor mye data som mangler for hver variabel, noe som er viktig informasjon når vi skal velge imputeringsmetoder eller eventuelt vurdere å fjerne variabler med høyt fravær av data. Her ser vi at `dnr_dag` og flere fysiologiske verdier har et høyt antall manglende verdier.



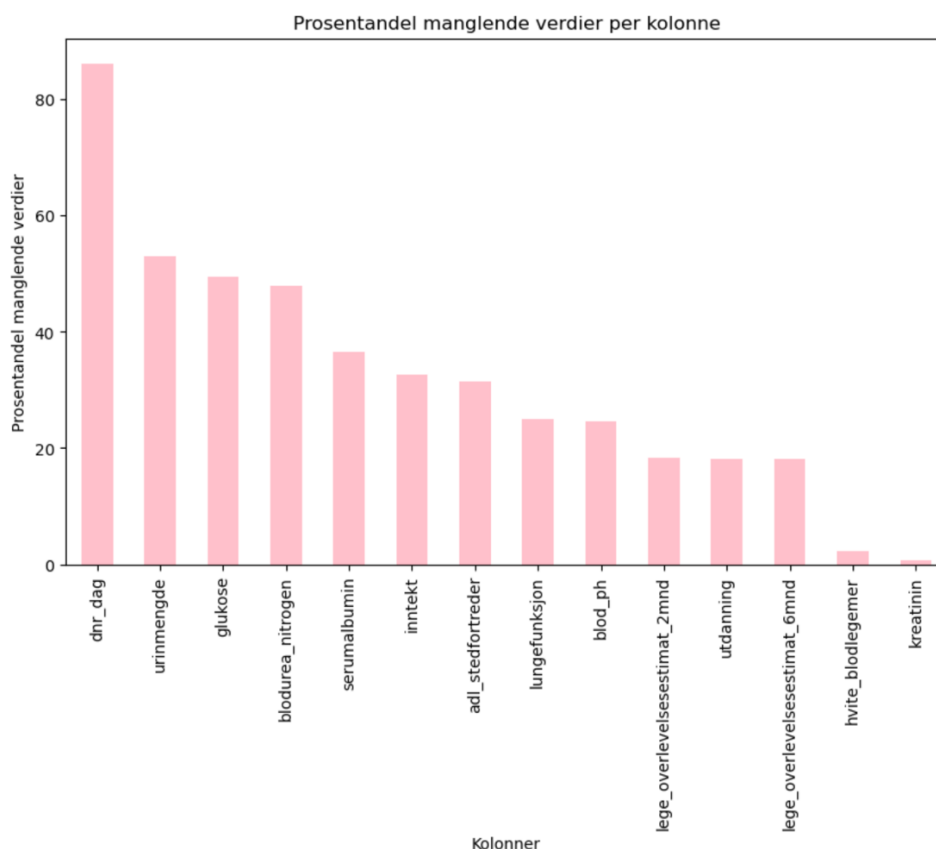
Figur 19: Heatmap over manglende verdier

DEL 2: Modelling og prediksjon

I denne delen av prosjektet er målet å bruke den bearbeidede dataen til å utvikle en maskinlæringsmodell som kan predikere oppholds-lengde for pasienter. Prosessen begynner med valg av en passende modell og håndtering av manglende data, da dette har stor betydning for modellens pålitelighet. Den utvalgte modellen vil deretter implementeres på en nettside som gjør det mulig å predikere oppholds-lengde, basert på input av ulike pasientvariabler.

Håndtering av manglende data

Håndtering av manglende data er avgjørende for å kunne utvikle en robust og pålitelig modell. Uten en god strategi for manglende verdier risikerer vi at modellen får redusert presisjon og generaliseringsevne, da ufullstendig data kan påvirke både treningsprosessen og de endelige prediksjonene. Gjennom tidligere utforskende dataanalyse observeres det en del fysiologisk data med manglende verdier og for `dnr_dag`. For å få en oversikt over disse verdiene igjen, visualiseres de på nytt.



Figur 20: Prosent over manglende verdier

For de fysiologiske variablene har vi standardverdier tilgjengelig, som settes inn for å sikre konsistens og realisme i dataene. Når det gjelder blod-ph, settes normalverdien inn som erstatning for manglende verdier. Dette gjøres for å opprettholde en biologisk realistisk verdi og unngå ekstreme verdier som kan oppstå ved andre typer imputering.

Videre har `dnr_dag` over 80 % manglende verdier, noe som gjør det utfordrende å impute på en pålitelig måte uten å tilføre kunstige data som kan påvirke modellens prediksjoner. `dnr_dag` er en funksjonsvariabel som angir dagen for en DNR-ordre (Do Not Resuscitate). Verdien kan være negativ dersom ordren ble gitt før studieperioden, 0 ved innleggelse, og NA dersom den ble gitt etter innleggelse. Ettersom prosjektets mål er å predikere oppholdslengde basert på informasjon tilgjengelig ved innleggelse, vurderes `dnr_dag` som en variabel som kan gi en urealistisk fordel i modellen. Særlig fordi denne variabelen ofte er assosiert med alvorlige pasientforløp og kortere opphold. Ved å inkludere `dnr_dag` risikerer vi at modellen «lærer» fra etterfølgende beslutninger snarere enn fra pasientens tilstand ved innleggelsen. For å unngå at modellen blir påvirket av faktorer som ikke er tilgjengelige tidlig i pasientforløpet og for å sikre bedre generaliserbarhet, fjernes `dnr_dag` fra datasettet."

Negative verdier

Under den utforskende dataanalysen oppdages uventet negativ verdi i variabelen `alder`, noe som ga oppmerksomhet på mulige andre urealistiske verdier. Derfor utvides analysen og det forekommer negative verdier i oppholdslengde, som heller ikke gir mening da denne skal representere antall dager på sykehus. For å sikre dataenes kvalitet fjernes observasjoner der `alder` og oppholdslengde hadde negative verdier. Siden det var svært få rader med negativ `alder`, utgjør dette ikke en stor forskjell for datagrunnlaget. Dette gir oss et datasett med kun realistiske verdier, noe som er viktig for modellens pålitelighet

Feature engineering

I løpet av den utforskende dataanalysen oppdages høy korrelasjon mellom flere variabler som vurderer pasientens alvorlighetsgrad og overlevelsesestimat. For eksempel viste både overlevelsesestimat og `lege_overlevelsesestimat` for 2 og 6 måneder en sterk korrelasjon, noe som indikerer at disse variablene måler lignende aspekter ved pasientens helsetilstand og forventede utvikling. Høy korrelasjon mellom slike variabler kan føre til redundans i datasettet, noe som øker kompleksiteten og kan påvirke modellens ytelse.

For å håndtere denne redundansen og skape en mer kompakt og informativ datastruktur, kombineres variablene til gjennomsnittlige verdier. Dette gir to nye variabler,

combined_overlevelsesestimat_2mnd og combined_overlevelsesestimat_6mnd, som fanger opp essensen av begge målinger uten å overvekte dem. Dette er en fordel for modellen, da det reduserer risikoen for overfitting som kan oppstå ved bruk av høyt korrelerte variabler.

I tillegg kombineres fysiologisk_score og apache_fysiologisk_score til en gjennomsnittlig variabel combined_fysiologisk_score, basert på en lignende observasjon om høy korrelasjon. Denne sammenslåingen gir en samlet indikator på pasientens fysiologiske tilstand uten unødvendig gjentakelse.

Til slutt fjernes variablene dnr_status og blodurea_nitrogen for å forbedre modellens generaliserbarhet og relevans. Analysen viste en negativ korrelasjon mellom dnr_status og oppholdslengde, noe som antyder at DNR-beslutninger ofte tas for pasienter med kort forventet opphold. dnr_status reflekterer kliniske beslutninger som kan skape prognostisk skjevhet og mulig etisk utfordrende bias, siden DNR-ordre er nært knyttet til alvorlig sykdom og økt sannsynlighet for sykehusdød. Siden modellens mål er å predikere oppholdslengde basert på pasientens fysiologiske og demografiske data, vurderes dnr_status som mindre relevant og potensielt misvisende for denne oppgaven.

Fra den utforskende dataanalysen oppdages en sterk korrelasjon mellom kreatinin og blodurea nitrogen, der høyere nivåer av kreatinin sammenfaller med høyere nivåer av blodurea nitrogen. Siden begge variabler måler nyrefunksjon, velges det å beholde kun kreatinin som en standardmåling for å unngå redundans og redusere kompleksiteten i datasettet. En alternativ tilnærming kunne vært å beregne en ratio mellom kreatinin og blodurea nitrogen, men på grunn av begrenset domenekunnskap velges en enklere løsning.

Modellering og model valg

For dette prosjektet brukes modellene Linear Regression, Random Forest, Gradient Boosting og Extra Trees for å predikere oppholdslengde. I tillegg trenes en enkel grunnlinjemodell, DummyRegressor, som alltid predikerer gjennomsnittet for oppholdslengde. Denne grunnlinjemodellen fungerer som et referansepunkt. For at de avanserte modellene skal anses som effektive, må de ha en lavere RMSE enn denne grunnlinjen.

Modellene trenes både på den opprinnelige datastrukturen og på data som er feature-engineered for å se om de nye variablene bidrar til forbedret ytelse. Ved å teste modellen på begge datasett kan vi vurdere om feature engineering tiltakene faktisk forbedrer prediksjonene, eller om den opprinnelige datastrukturen gir tilstrekkelig nøyaktighet. Sammenligningen mellom disse datasettene gir verdifull innsikt i hvilken

datarepresentasjon som best støtter modellens mål om å predikere oppholdslengde mest presist.

Evaluerings og måling

For å evaluere modellene sammenlignes den predikerte oppholdslengden med den faktiske oppholdslengden i valideringssettet. Evalueringen skjer gjennom RMSE (root mean squared error), som måler forskjellen mellom predikert og faktisk verdi. En lavere RMSE indikerer at modellen predikerer oppholdslengde mer presist. Den beste modellen blir dermed den som oppnår lavest RMSE, noe som gir en enkel og presis indikator på modellens ytelse.

Hyperparameteroptimalisering

For å forbedre modellens ytelse benyttes hyperparameteroptimalisering på mer komplekse modeller som Random Forest, Gradient Boosting og Extra Trees. Ved å variere antallet estimators 50, 100 og 150 kan vi finne den optimale balansen mellom modellens nøyaktighet og kompleksitet. Et høyere antall estimators kan ofte gi bedre ytelse, men det øker også treningskostnaden og risikoen for overfitting, spesielt hvis modellen tilpasser seg spesifikke mønstre i treningsdataene.

- Random Forest: Ved bruk av `n_estimators`-parametrene (50, 100, og 150) testes hvor mange beslutningstrær som gir best balanse mellom ytelse og effektivitet. Random Forest håndterer variabler med manglende data robust og kan fange opp komplekse sammenhenger.
- Gradient Boosting: For denne modellen vurderes også `n_estimators`-verdier. Boosting-modeller bygger sekvensielt på svakere estimer for å gradvis forbedre prediksjonene, noe som gjør dem ideelle for komplekse relasjoner i dataene.
- Extra Trees: Likt som Random Forest, tester `n_estimators`-verdier for Extra Trees-modellen. Extra Trees er en ensemble metode som bruker flere beslutningstrær for å forbedre prediksjoner. Den kan ofte være raskere enn Random Forest og gir en effektiv tilnærming til modellering, spesielt når man arbeider med store datasett med manglende verdier.

Imputeringsteknikker

For å håndtere manglende data, brukes imputering. Imputering av data betyr å legge inn estimerte verdier for manglende data i datasettet. Dette er nødvendig for å kunne trene og predikere med maskinlæringsmodeller, da modellene ikke håndterer manglende data selv. Manglende data behandles med tre ulike imputeringsteknikker: mean, median, og KNN (K-Nearest Neighbors). Valget av riktig imputering er viktig for å unngå at modellen påvirkes av data som ikke er representativt for populasjonen.

- Mean Imputering: Brukes som en enkel metode som fyller manglende verdier med gjennomsnittet av variabelen. Denne metoden er effektiv og rask, men kan påvirkes av ekstreme verdier.

- Median Imputering: Medianen er mer robust for variabler med skjev fordeling eller uteliggere, og kan være et bedre valg enn gjennomsnittet når slike variabler forekommer.
- KNN Imputering: Denne metoden fyller inn manglende verdier ved å bruke verdiene til de nærmeste naboene, noe som kan gi mer presise estimater for variabler med komplekse mønstre. KNN-imputering er nyttig når variabelen har naturlige sammenhenger med andre variabler i datasettet.

Ved å sammenligne modellene med hver av disse imputeringsteknikkene, kan vi finne en optimal kombinasjon som gir den mest pålitelige prediksjonen av oppholdslengde. Modellen med lavest RMSE etter hyperparameteroptimalisering og imputering blir valgt som den beste, og gir dermed et solid grunnlag for videre prediksjon av pasienters oppholdslengde.

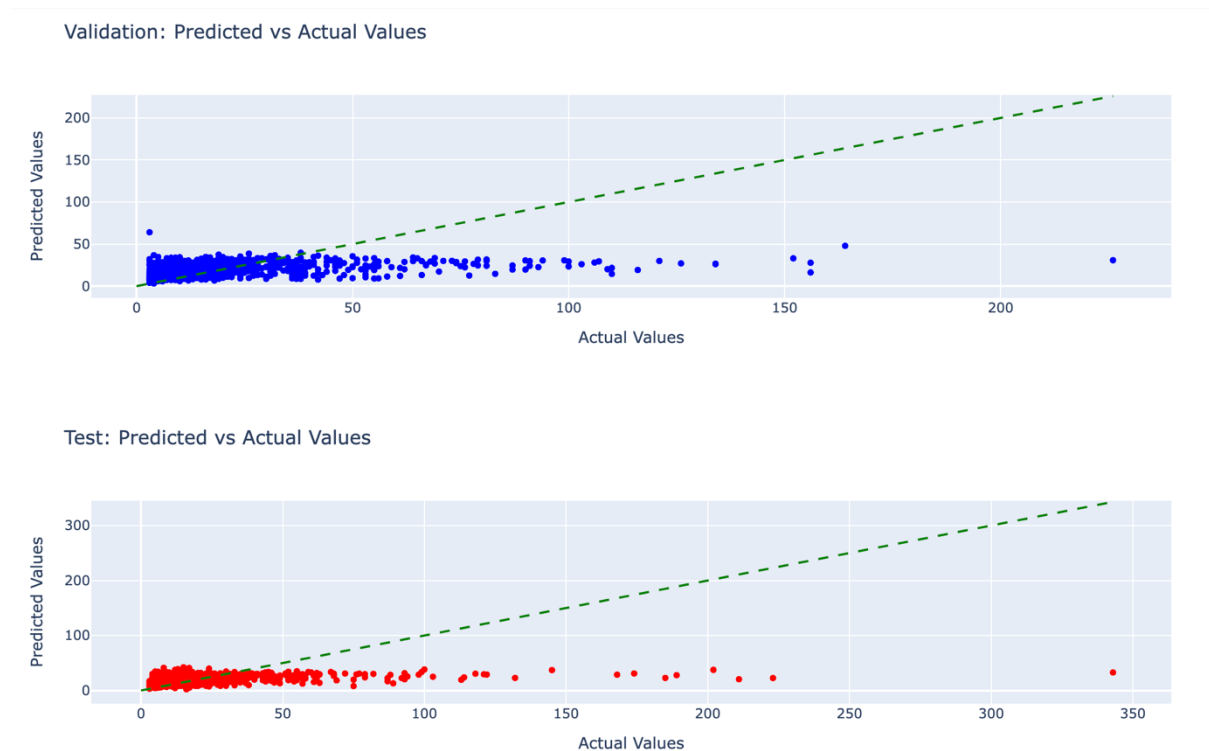
Den beste modellen var Gradient Boosting med median imputer og 50 estimators, trent på det originale datasettet uten feature engineering. Modellen oppnådde en RMSE på 19.752 på valideringsdata, som er lavere enn grunnlinjemodellens RMSE på 21.49. Denne lavere RMSE viser at Gradient Boosting modellen predikerer oppholdslengde mer nøyaktig enn den enkle gjennomsnittsprediksjonen, og at de opprinnelige variablene var tilstrekkelige for gode prediksjoner, uten ytterligere feature engineering.

Generaliseringsevne

Modellens generaliseringsevne evalueres ved å undersøke ytelsen på testdata. RMSE for testdata ble 22.603, noe som er høyere enn RMSE for valideringsdata på 19.752. Denne økningen i RMSE indikerer at modellen presterer litt dårligere på testdata enn på valideringsdata, noe som kan ha flere årsaker.

For det første kan modellen ha blitt noe overfitted til trenings- og valideringsdataene (overfitting), noe som innebærer at den har tilpasset seg spesifikke mønstre i disse datasettene som ikke nødvendigvis finnes i testdataene. Overfitting kan føre til svakere ytelse når modellen møter nye data som testsettet, som representerer helt nye observasjoner. Grafene viser også at prediksjonene er konsentrert rundt de lavere verdiene for oppholdslengde, mens faktiske verdier i høyere områder (over 100 dager) ofte er undervurdert.

I tillegg kan variasjoner i distribusjonen mellom trenings og testdataene gjøre det utfordrende for modellen å generalisere nøyaktig. Testdata kan inneholde eksempler med svært lange oppholdslengder, noe som modellen har problemer med å forutsi nøyaktig, som er vist i de spredte punktene på grafene. Dette fører til større avvik mellom predikert og faktisk verdi i testsettet, noe som resulterer i en høyere RMSE. Den høyere feilen på testdata tyder på at modellen har en viss begrensning i generaliseringsevnen, og dette kan være et område for videre forbedringer.



Figur 21: Predikert vs Faktisk verdier

Feature importance

Målet med feature importance er å identifisere hvilke variabler fra datasettet som er mest relevante for modelleringen, slik at vi kan fokusere på de mest betydningsfulle faktorene og redusere unødvendig kompleksitet. Å inkludere for mange features i modellen øker risikoen for overfitting, der modellen tilpasser seg for godt til treningsdataene og dermed mister evnen til å generalisere godt til nye data. Overfitting innebærer at modellen lærer detaljer og støy spesifikke for treningssettet, som kan redusere nøyaktigheten på testsettet. Dette kan være noe av årsaken til at modellen vår oppnådde høyere RMSE på testdata enn på valideringsdata, som nevnt i forrige avsnitt. Ved å bruke feature importance kan vi identifisere de mest verdifulle variablene og potensielt redusere modellens kompleksitet, noe som kan bidra til bedre generalisering og mer pålitelige prediksjoner på nye data.

For å identifisere de viktigste variablene for modellens prediksjoner, beregnes feature importance for den beste modellen. Dette gjorde det mulig å rangere variablene etter deres påvirkning på prediksjonene, og deretter teste modellen med ulike antall av de høyest rangerte variablene. Målet med denne prosessen var å redusere risikoen for overfitting ved å fjerne mindre viktige variabler som kunne tilføre støy og begrense modellens generaliseringsevne. Resultatet viste at den beste modellen er trent med de 28 mest relevante variablene. Denne modellen oppnådde en Validation RMSE på 19.838

og en Test RMSE på 22.574. Selv om Validation RMSE økte noe (med 0.086) sammenlignet med modellen uten feature selection, er Test RMSE fortsatt lavere enn tidligere. Dette bekrefter at bruk av feature importance forbedret modellens nøyaktighet og generaliseringsevne. Den lille økningen i Validation RMSE tyder på at reduksjonen i features har minimal effekt på modellen, samtidig som den lavere Test RMSE. Dette indikerer at modellen med færre, men mer relevante variabler gir bedre og mer pålitelige prediksjoner på nye data.

Modell valg for nettside

For å lage en brukervennlig nettside for prediksjon av oppholdslengde, har jeg valgt å bruke de 10 mest relevante variablene, da dette forenkler inputprosessen for brukeren. Forskjellen i RMSE på testdata mellom modellen med topp 28 features (22.574) og topp 10 features (22.699) er svært liten og utgjør mindre enn en dags forskjell i prediksjon, noe som gjør det praktisk å bruke en enklere modell uten betydelig tap i nøyaktighet. Jeg lagrer derfor to modeller: en med topp 28 features, som brukes til å generere prediksjoner i en CSV-fil, og en med topp 10 features for nettsiden.

For input på nettsiden kreves fullstendig data for alle variabler, da manglende verdier vil resultere i at modellen ikke kan produsere en prediksjon. I selve modellen som brukes i data science-prosjektet, benyttes median imputer for å fylle ut manglende verdier, noe som sikrer robusthet og nøyaktighet der data kan være ufullstendig. På nettsiden har jeg imidlertid valgt å kreve full input for å holde prediksjonen enkel og brukervennlig, og dermed unngå potensielle feilkilder som kan oppstå med sanntids imputering.

DEL 3: Drøfting av resultater og mulige forbedringer

Den beste modellen gir en validerings-RMSE på 19.838, noe som er lavere enn grunnlinjemodellens RMSE på testdata (21.49). Dette tyder på at modellen har klart å lære mønstre som gir bedre prediksjon på valideringssettet enn det en enkel grunnlinjemodell klarer. Likevel er RMSE på testdata for den beste modellen høyere (22.574), noe som tyder på at modellen ikke generaliserer optimalt til nye data, til tross for forbedring på valideringsdata. Dette kan skyldes overfitting eller kompleksiteten i de medisinske dataene, som krever spesialkunnskap for å fange opp subtile sammenhenger og effekter.

Dette viser at den avanserte modellen ikke presterer bedre enn en enkel grunnlinjemodell når det gjelder nøyaktighet. Ettersom dette er et svært omfattende datasett, med komplekse medisinske data, et felt jeg ikke har dyp kjennskap til, var jeg forberedt på at modellen kunne ha utfordringer med å oppnå høy nøyaktighet. I tillegg er dette mitt første semester med et data science fag, og det å arbeide med så store og komplekse data er en krevende oppgave. Jeg forventet derfor en modell som kanskje ikke ville være så robust og godt tilpasset.

Når det er sagt, hadde jeg kanskje forventet at modellen kunne prestere litt bedre enn den enkle grunnlinjemodellen, spesielt på kortere oppholdslengder der variasjonene er mindre ekstreme. Det ville vært interessant om modellen i det minste hadde klart å fange opp noen av de mer åpenbare mønstrene i dataene, som visse sammenhenger mellom fysiologiske variabler og oppholdslengde.

Jeg ser at modellen har spesielt utfordringer med å generalisere ved svært lange oppholdslengder, der prediksjonene ofte har store avvik fra de faktiske verdiene. Ved å analysere prediksjonene i predictions.csv, som bruker den beste modellen til å estimere oppholdslengde for pasienter, ser jeg at den for det meste gir prediksjoner på under 30 dager. Dette tyder på at modellen sliter med å fange opp faktorer som bidrar til lengre opphold, og dermed undervurderer disse tilfellene. Denne utfordringen var også tydelig i visualiseringene av test- og valideringsprediksjonene, hvor modellen viste større avvik ved lengre opphold. Selv om modellen kan gi innsikt for analyser og planlegging, viser resultatene at den kan være mindre egnet til presise kliniske beslutninger hvor nøyaktighet er kritisk.

Med mer tid og ressurser kunne flere tiltak blitt utforsket for å forbedre modellen ytterligere. For det første kunne økt domenekunnskap vært nyttig. Ved å sette seg dypere inn i medisinske sammenhenger kunne jeg kanskje identifisert nye viktige variabler. Dette kunne også inkludere en mer detaljert forståelse av hvordan ulike sykdommer og behandlinger påvirker oppholdslengde. For eksempel var sykdommen ARF/MOSF med

sepsis topp en feature i modellen. Med mer innsikt i hvordan denne tilstanden påvirker oppholdslengde, kunne jeg kanskje utviklet mer spesifikke variabler eller justeringer for pasienter med denne sykdommen.

I tillegg kunne mer fokus på dataforberedelse og feature engineering forbedret modellens ytelse. For eksempel ved å håndtere outliers, enten ved å fjerne dem eller transformere verdiene, kunne jeg redusert deres innvirkning på modellen og potensielt forbedret generaliseringsevnen. I denne analysen har jeg imidlertid valgt å ikke gjøre noe med outliers, da jeg ikke er helt sikker på den beste måten å håndtere dem på og hva som ville være mest hensiktsmessig. Outliers kan representere viktige kliniske tilfeller, men de kan også introdusere støy som påvirker modellens ytelse. Hvordan disse bør håndteres krever en mer grundig vurdering mener jeg.

Videre kunne jeg laget sammensatte variabler som kombinerer ulike helseindikatorer, for eksempel en kombinert score som måler samlet helse og risikonivå for pasienten. Til slutt kunne en grundigere hyperparametertuning ha optimalisert modellens ytelse ytterligere, spesielt med tanke på valg av parametere som best passer til datasettene vi arbeider med. Ved å kombinere tekniske forbedringer som disse med økt innsikt i medisinske faktorer, kunne modellen potensielt gi mer nøyaktige og generaliserbare prediksjoner, også i tilfeller med mer ekstreme utfall.

Bibliografi

- geeksforgeeks. (2023, mai 02). *geeksforgeeks*. Retrieved from ML | Feature Mapping: <https://www.geeksforgeeks.org/feature-mapping/>
- geeksforgeeks. (2024, november 02). Retrieved from One Hot Encoding in Machine Learning: <https://www.geeksforgeeks.org/ml-one-hot-encoding/>
- geeksforgeeks. (2024, mars 11). *geeksforgeeks*. Retrieved from ML | Underfitting and Overfitting: <https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>
- geeksforgeeks. (2024, august 08). *geeksforgeeks*. Retrieved from Machine Learning Models: <https://www.geeksforgeeks.org/machine-learning-models/>
- Harrel, F. (2023, september14). *UC Irvine Machine Learning Repository*. Retrieved from Support 2: <https://archive.ics.uci.edu/dataset/880/support2>
- Lewis III, J. L. (2023, juli). *MSD Manual Consumer Version* . Retrieved from Overview of Acid-Base Balance: <https://www.msdmanuals.com/home/hormonal-and-metabolic-disorders/acid-base-balance/overview-of-acid-base-balance>
- Wang, Y., Xu, X., Shi, S., Gao, X., Li, Y., Wu, H., . . . Zhang, B. (2023, september 14). *European Journal of Medical Research*. Retrieved from Blood urea nitrogen to creatinine ratio and long-term survival in patients with chronic heart failure: <https://eurjmedres.biomedcentral.com/articles/10.1186/s40001-023-01066-x>

